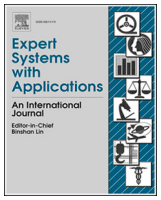
Expert Systems With Applications 319 (2026) 132112



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa





A smooth reinforcement learning method for trajectory tracking and collision avoidance of wheeled vehicle

Liangfa Chen [a,b,1], Xujie Song [c,1], Junjie Zhao [a], Wenxuan Wang [d], Yinuo Wang [d], Liming Xiao [a], Shengbo Eben Li [d], Fei Ma [a], Jingliang Duan [a,*]

[a] School of Mechanical Engineering, University of Science and Technology Beijing, 100083, Beijing, China
[b] Sunrising AI, 100084, Beijing, China
[c] University of Michigan, Ann Arbor, MI, 48109, USA
[d] School of Vehicle and Mobility, Tsinghua University, 100084, Beijing, China

ARTICLE INFO

Keywords:
Intelligent vehicle
Trajectory tracking
Reinforcement learning
Action smoothing

ABSTRACT

Reinforcement learning (RL) plays a pivotal role in solving complex tasks such as vehicle trajectory tracking and collision avoidance. However, RL-derived control policies often suffer from a lack of smoothness, leading to abrupt changes in control actions in response to minor state variations, which can accelerate component degradation and lead to system failures in real-world scenarios. To mitigate this, we propose a novel neural network architecture named MoLiNet, designed to smooth actions across multiple dimensions. MoLiNet dynamically regulates individual Lipschitz constants for each component of the action vector using a trainable parameterized network, thereby improving control smoothness. Building on this, we merge MoLiNet with distributional soft actor-critic (DSAC) to create a new RL algorithm, MoLiNet-DSAC. This algorithm incorporates an L2 regularization term linked to the Lipschitz constants of action components into the policy optimization objective, striving for an optimal balance between control efficacy and smoothness. Experimental trials on actual wheeled vehicles reveal that MoLiNet-DSAC not only improves action smoothness but also significantly enhances position tracking precision, surpassing conventional smooth network architectures like LipsNet.

1. Introduction

In recent years, autonomous driving has received widespread attention from academia and industry (Li, 2023). Within this field, trajectory tracking and collision avoidance emerge as vital and challenging aspects of autonomous vehicle technology (Duan et al., 2024). These challenges are primarily attributed to the need for managing intricate system constraints and maintaining stringent real-time control. Existing online methods, like MPC and iLQR, generally require solving the control law for each control cycle online (Grüne et al., 2017; Ma et al., 2022). However, this approach often fails to meet the feasibility and real-time requirements when computational resources are limited.

Leveraging the powerful fitting abilities of neural networks (NNs), reinforcement learning (RL) based control has seen a surge in research and practical applications (Duan et al., 2024; Elhaki et al., 2022; Wang et al., 2024). Lu et al. (2024) utilized parallel control and DDPG (Lillicrap et al., 2015) to achieve comfortable autonomous driving for path tracking. He et al. (2022) developed an adversarial RL algorithm designed to improve the safety and stability of lane-changing maneuvers in autonomous vehicles by reducing the discrepancy between the lane change policy and the policy impacted by optimal adversarial observation perturbations. This approach involves initially learning a policy offline followed by its online deployment for practical applications. Significantly, RL has shown substantial promise in managing tasks related to trajectory tracking and collision avoidance (Li et al., 2025; Udekwe et al., 2024; Wu et al., 2026). For instance, Luo et al. (2022) integrated a collision penalty into the policy objective function, enabling effective vehicle trajectory tracking and static obstacle avoidance. Peng et al. (2022) developed a separated proportional-integral Lagrangian algorithm, formulating trajectory tracking and collision avoidance as a probability-constrained optimal control problem. Utilizing the PID-Lagrangian methodology, they have successfully learned control policies

* Corresponding author.
E-mail addresses: chenliangfa@sunrisingai.com (L. Chen), xjsong@umich.edu (X. Song), zhaojunjie@xs.ustb.edu.cn (J. Zhao), wang-wx18@tsinghua.org.cn (W. Wang), wyn23@mails.tsinghua.edu.cn (Y. Wang), xlming@xs.ustb.edu.cn (L. Xiao), lishbo@tsinghua.edu.cn (S. Eben Li), yeke@ustb.edu.cn (F. Ma), duanjl@ustb.edu.cn (J. Duan).
[1] These authors contributed equally.

https://doi.org/10.1016/j.eswa.2026.132112
Received 23 November 2025; Received in revised form 4 March 2026; Accepted 16 March 2026
Available online 19 March 2026
0957-4174/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

that ensure dynamic collision avoidance and maintain high performance during trajectory tracking tasks.

However, existing RL algorithms often face the issue of action non-smoothness, particularly under noisy conditions. Completely mitigating noise in real-world control systems is a challenging task, and even minor disturbances can lead to significant fluctuations in the actions produced by NNs. Such variability in control actions can precipitate accelerated wear on mechanical components and introduce safety hazards (Yu et al., 2021). Mysore et al. (2021) observed that direct filtration of the output actions from a trained policy network will change the dynamic response of the system and violate the Markov hypothesis, thereby critically affecting controllability. To strike a balance between control optimality and smoothness, it is imperative to curb action fluctuation during policy training. Kobayashi (2022) introduced a smoothness-related penalty term into the loss function of the policy network. However, this approach demands extensive parameter tuning for optimal performance. Shen et al. (2020) aimed to enhance policy smoothness by integrating disturbance noise through adversarial policy design, though this method adds complexity to the learning process.

From a theoretical standpoint, action fluctuation is influenced by the smoothness of the NN landscape, with the Lipschitz constant serving as an indicator of this smoothness. Contrasting with methods from Kobayashi (2022) and Shen et al. (2020) that modify the RL algorithm to achieve Lipschitz constant constraints, recent studies have focused on embedding these constraints directly within the network structure. As an early innovator in this domain, Takase et al. (2022) tried to regulate the Lipschitz constant of policy NNs using spectral normalization (SN) at the layer level, which, while effective in reducing action fluctuation, often leads to significant performance trade-offs due to the imposition of a global Lipschitz constant. In response to these challenges, Song et al. (2023) proposed the LipsNet structure that dynamically adjusts the local Lipschitz constant of the policy network, facilitating a state-dependent modulation of smoothing capabilities. However, these solutions are predominantly tailored to tasks with one-dimensional action spaces. In scenarios involving multi-dimensional control, a single Lipschitz constant is insufficient to capture the diverse control characteristics across different actions, leading to challenges in concurrently optimizing the smoothness across each component of control vectors and the overall control performance. This is particularly problematic in tasks like trajectory tracking and collision avoidance, which demand coordinated control over multiple action types, such as steering for lateral movement and acceleration or velocity for longitudinal movement.

To overcome these challenges, this study introduces a new network architecture, termed MoLiNet. This innovative structure is tailored to adaptively smoothen multi-dimensional control actions, making it particularly suitable for tasks such as vehicle trajectory tracking and collision avoidance control. Our main contributions can be summarized as follows.

1. A NN structure with multi-dimensional action smoothing capability, named MoLiNet, is proposed to enhance control smoothness. It adaptively outputs individual Lipschitz constants for each element of the multi-dimensional action vector through a trainable parameterized network, thus ensuring distinct local Lipschitz continuity constraints for each action component. Further, we extend it to stochastic policies so that it can be used in most existing RL algorithms without any modification.
2. To facilitate the acquisition of a smooth control policy, we further propose an innovative RL algorithm called MoLiNet-DSAC, which integrates MoLiNet with the distributional soft actor-critic (DSAC) (Duan et al., 2021, 2025). This algorithm features a critic for learning the distributional value function and a MoLiNet-based actor for enacting a smooth control policy. To enhance control effectiveness while preserving smoothness, an L2 regularization term related to the Lipschitz constant is added to the policy optimization goal. This encourages smaller Lipschitz constants for each action component, thereby balancing performance improvement with control smoothness.
3. The policy developed through MoLiNet-DSAC is implemented on actual wheeled vehicles for executing trajectory tracking and collision avoidance tasks. The outcomes reveal that in comparison to DSAC which utilizes the pre-existing smooth network architecture known as LipsNet (Song et al., 2023), our approach not only enhances action smoothness but also significantly boosts position tracking precision by over three times.

The paper is organized as follows: Section 2 introduces the preliminaries of RL and the metric of policy smoothness. Section 3 introduces the network architecture of MoLiNet and the MoLiNet-DSAC algorithm. Section 4 assesses the performance of MoLiNet-DSAC in comparison to other methods in a vehicle trajectory task. Section 5 evaluates the real-world performance of MoLiNet-DSAC in a wheeled vehicle obstacle avoidance experiment. Section 6 concludes the paper.

2. Preliminary

2.1. Infinite horizon optimal control task

In general, a vehicle trajectory tracking task can be formulated as an infinite-time horizon optimization control problem:

$$\max_{a_t,\dots,a_{t+i},\dots\in\mathcal{A}} \sum_{i=0}^{\infty} \gamma^i r(s_{t+i}, a_{t+i}) \tag{1}$$

$$\text{s.t.} \quad s_{t+i+1} = f(s_{t+i}, a_{t+i}), \quad i \in \mathbb{N},$$

where γ is the discount factor, $r(s_t, a_t)$ is the reward function, $s_t \in S \subset \mathbb{R}^n$ is the system state, $a_t \in \mathcal{A} \subset \mathbb{R}^m$ is the control input, and $f : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$ is the system dynamics.

From an optimization perspective, the goal of traditional RL is to find an optimal control sequence $\{a_t^*, \dots, a_{t+i}^*, \dots\} \sim \pi^*$ that maximizes the infinite-step cumulative return (1). Here, π^* represents the optimal control policy, a_t^* is the optimal control action at time t. Given the initial state s_t, the value function Q are defined as

$$Q_\pi(s_t, a_t) \triangleq \mathbb{E}_\pi \left[\sum_{i=0}^{\infty} \gamma^i r\left(s_{t+i}, a_{t+i}\right) \right], \tag{2}$$

where value function $Q(\cdot)$ represents the cumulative expected return of the agent by executing the policy $\pi(\cdot)$ to infinity time step from the state s_t with a_t as an initial action.

The policy iteration framework is used to optimize function Q and π alternately, gradually converging towards the optimal policy π^*. Policy iteration is divided into two parts: policy evaluation and policy improvement. The policy evaluation process estimates the Q-value of policy π using

$$Q_\pi(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_\pi \left[Q_\pi(s_{t+1}, a_{t+1}) \right]. \tag{3}$$

The policy improvement step seeks a better policy π_{new} by maximizing the current Q-value estimation $Q_{\pi_{\text{old}}}$:

$$\pi_{\text{new}}(s_t) = \arg\max_{a_t} \left[Q_{\pi_{\text{old}}}(s_t, a_t) \right]. \tag{4}$$

2.2. Lipschitz constraints in NNs

Assuming function $\pi : \mathbb{R}^n \to \mathbb{R}^m$ is local K-Lipschitz continuous function over $\mathcal{X}$ where $\mathcal{X} \subset \mathbb{R}^n$, then there exists a constant $K > 0$ that satisfies:

$$\|\pi(x_1) - \pi(x_2)\| \le K \cdot \|x_1 - x_2\|, \forall x_1, x_2 \in \mathcal{X}, \tag{5}$$

where the smallest K that satisfies this inequality is termed the local Lipschitz constant over $\mathcal{X}$. Lipschitz continuity ensures the stability of a function with respect to input perturbations, meaning that the function's output changes slowly relative to changes in its input.

Consider the kth layer of the network $f_k = a^k(g(x)) = a^k(W^k x)$, where a^k is the activation function, $g(x)$ is the linear layer, and W^k is the weight matrix. The Lipschitz norm $\|g\|_{\text{Lip}}$ is equal to $\sup_x \sigma(\nabla g(x))$, where $\sigma(W)$ represents the spectral norm of matrix W, i.e., its maximum singular value. For activation functions like ReLU, Sigmoid, and Tanh, the Lipschitz norm $\|a^k\|_{\text{Lip}}$ is 1. Therefore, the fully connected network π satisfies:

$$\|\pi\|_{\text{Lip}} \leq \prod_{k=1}^{n} \|a^k\|_{\text{Lip}} \|W^k\|_{\text{Lip}} = \prod_{k=1}^{n} \sigma(W^k). \tag{6}$$

In earlier research (Takase et al., 2022), spectral normalization (SN) was used to constrain the spectral norm of each layer's matrix. This ensures π adheres to 1-Lipschitz continuity. However, applying constraints at the layer level can cause significant performance degradation.

Nevertheless, Lipschitz continuity itself plays a crucial role in governing the sensitivity and stability of the policy. It not only provides a structural bound on the network but also has important implications for policy behavior and optimization stability. Specifically, Lipschitz continuity bounds the sensitivity of the policy mapping with respect to its input. For a differentiable mapping, a finite Lipschitz constant implies that the operator norm of its Jacobian is bounded almost everywhere, i.e.,

$$\|\nabla_s \pi(s)\| \leq \|\pi\|_{\text{Lip}}, \tag{7}$$

almost everywhere.

Therefore, applying Eq. (5) to consecutive states s_t and s_{t-1} yields a deterministic upper bound

$$\|\pi(s_t) - \pi(s_{t-1})\| \leq \|\pi\|_{\text{Lip}} \|s_t - s_{t-1}\|. \tag{8}$$

Such bounded sensitivity not only provides a strict upper bound on temporal action variation but also effectively mitigates the risk of gradient amplification during backpropagation, thereby improving the numerical stability of policy optimization.

2.3. Evaluation metric for action fluctuation

To quantitatively measure the fluctuation of control actions, Chen et al. (2021) introduced a metric for discrete action spaces. This metric was further extended by Song et al. (2023) to encompass continuous action spaces, leading to the formulation of action fluctuation ratio, defined as

$$\xi(\pi) = \mathbb{E}_{\tau \sim \rho_\pi} \left[\frac{1}{T} \sum_{t=1}^{T} \|a_t - a_{t-1}\| \right], \tag{9}$$

where ρ_π is the distribution of state-action trajectory induced by policy π, T represents the termination time of trajectories, a_t and a_{t-1} represent the control actions at current and previous time steps, respectively. Notably, a lower value of $\xi(\pi)$ implies a higher level of smoothness for the actions yielded by policy π.

3. Methodology

In this section, we propose a smooth network structure for systems with multi-dimensional control actions, termed as MoLiNet. We highlight that MoLiNet is highly flexible and can be seamlessly adopted into various RL algorithms without requiring any modifications. As concrete examples, we combine MoLiNet with DSAC, a state-of-the-art model-free RL algorithm, and propose a smooth DSAC algorithm (MoLiNet-DSAC).

3.1. MoLiNet: Multi-output Lipschitz network

In RL, parameterized functions are typically used to approximate both the Q-value function and the policy function. Thanks to the Universal Approximation Theorem (Hornik et al., 1989), neural networks (NNs) have become a predominant tool for policy modeling. However, NN-based policies often suffer from action fluctuations, particularly in control tasks with high-dimensional action spaces, such as robotic manipulations and drones, where ensuring reliable action smoothness and control performance is critically important. To independently constrain the fluctuation of each action component, we propose an innovative network architecture, termed MoLiNet, which adaptively regulates the Lipschitz constant for each action dimension, thereby significantly improving the overall smoothness and accuracy of control.

MoLiNet achieves this by producing distinct Lipschitz constants for each action dimension through a learnable constant network $K(s_t;\varphi) : \mathbb{R}^n \to \mathbb{R}^m$. This ensures that tailored constraints are applied to each individual action component generated by the original policy network $\pi(s_t;\omega) : \mathbb{R}^n \to \mathbb{R}^m$, where φ and ω represent the parameters of the constant network and the original policy network, respectively. Among these, the ith dimensional action output by MoLiNet, denoted as $\pi_s(s_t;\omega,\varphi)[i]$, satisfies

$$\pi_s(s_t;\omega,\varphi)[i] = K(s_t;\varphi)[i] \cdot \frac{\pi(s_t;\omega)[i]}{\|\nabla_{s_t} \pi(s_t;\omega)[i]\| + \epsilon}, \tag{10}$$

where $i \in \{1, \ldots, m\}$, $K(s_t;\varphi)[i]$ is the ith output of the Lipschitz network, $\|\cdot\|$ means the vector norm operation for the gradient vector $\nabla_{s_t} \pi(s_t;\omega)[i]$, and ϵ is a small positive constant. The output of MoLiNet is jointly determined by the constant network $K(s_t;\varphi)$ and the gradient norm of the original policy network $\pi(s_t;\omega)$. Distinct from conventional single policy networks, the Lipschitz continuity of the output function $\pi_s(s_t;\omega,\varphi)[i]$ in the MoLiNet architecture cannot be directly inferred from the continuity of $\pi(s_t;\omega)$. Based on (10), we can draw the following theorem, with a comprehensive proof provided in Appendix A:

Theorem 1. *Suppose the original policy network $\pi(s_t;\omega) : \mathbb{R}^n \to \mathbb{R}^m$ is a continuous NN with piecewise linear activation function. Assume that for $\forall s_t \in S$, the constant K_i is the scalar corresponding to $K(s_t;\varphi)[i]$, Then, for $\forall i$, within each piecewise interval, the function mapping from input s_t to the ith dimensional action $\pi_s(s_t;\omega,\varphi)[i]$, as specified by (10), exhibits local K_i-Lipschitz continuity.*

Theorem 1 suggests that MoLiNet is able to ensure local Lipschitz continuity for every element of the action vector, using the output of $K(s_t;\varphi)[i]$ as the Lipschitz constant. This enables adaptive continuity control across different state neighborhoods, which holds greater significance for real-world control applications. Specifically, this trainable Lipschitz parameter $K(s_t;\varphi)$ provides theoretical guarantees for both temporal smoothness and training stability through the following mechanisms. For temporal smoothness, in discrete-time control, the action difference between consecutive timesteps satisfies:

$$\|a_t - a_{t-1}\| = \|\pi_s(s_t) - \pi_s(s_{t-1})\| \leq \bar{K}(s_t;\varphi)\|s_t - s_{t-1}\|, \tag{11}$$

where $\bar{K}(s_t;\varphi) = \max_i K(s_t;\varphi)[i]$ represents the maximum Lipschitz constant across all action dimensions. Given that physical systems exhibit inertial constraints with bounded state evolution $\|s_t - s_{t-1}\| \leq L_{\text{dyn}}\Delta t$, MoLiNet provides a deterministic upper bound $\|a_t - a_{t-1}\| \leq \bar{K}(s_t;\varphi)L_{\text{dyn}}\Delta t$, mathematically suppressing high-frequency chattering. For robustness to perturbations, sensor noise ϵ with $\|\epsilon\| \leq \delta$ induces action perturbation bounded by:

$$\|\pi_s(s_t + \epsilon) - \pi_s(s_t)\| \leq \bar{K}(s_t;\varphi)\delta. \tag{12}$$

This ensures control stability under perception uncertainties. From a training perspective, the element-wise constraint $\|\nabla_{s_t} \pi_s[i]\| \leq K(s_t;\varphi)[i]$ acts as implicit gradient clipping, preventing gradient explosion and improving numerical conditioning during backpropagation.

MoLiNet, as defined by (10), integrates well with RL algorithms that utilize deterministic policies for policy approximation. However, when it comes to stochastic policies, the output is typically a distribution rather than a fixed value. To accommodate this, MoLiNet has been customized for stochastic policy applications, often modeled using Gaussian distributions as $\pi(\cdot|s_t;\omega) = \mathcal{N}\left(\mu(s_t;\omega), \sigma(s_t;\omega)^2\right)$, where $\mu(s_t;\omega)$ and $\sigma(s_t;\omega)$ represent the mean and standard deviation of the action distribution, respectively. In such cases, (10) is exclusively used to adjust

the local Lipschitz constant for the mean action $\mu_i(s_t;\omega)$, while leaving $\sigma(s_t;\omega)$ unchanged. The process concludes by merging the adjusted mean with the standard deviation to create the refined smoothed policy $\pi_s(s_t;\omega,\varphi)$, The comprehensive architecture of the MoLiNet is depicted in Fig. 1.

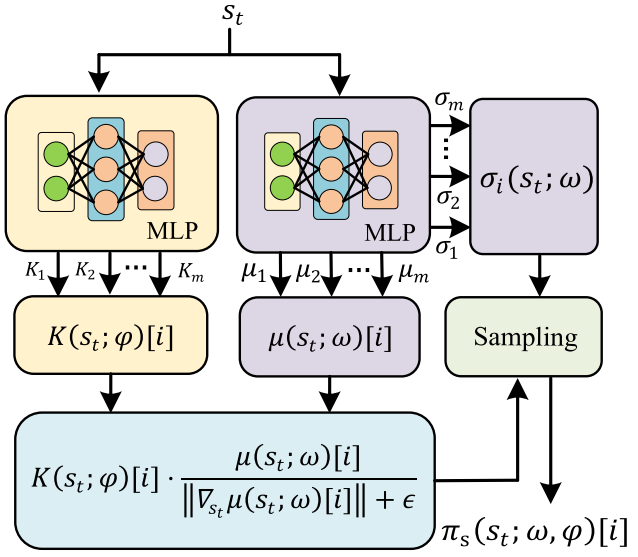


Fig. 1. MoLiNet structure. The input is s_t. The original policy $\pi(\cdot)$ is represented by the purple MLP network, while the Lipschitz constant network $K(\cdot)$ is portrayed by the yellow MLP network, with the output dimension aligned with that of $\mu(\cdot)$. It is noteworthy that the Softplus activation function is employed in the last layer of the K network, and derivative operations are required for $\mu(\cdot)$. The $\pi_s(\cdot)$ signifies the final output's smoothing policy after sampling.

3.2. RL with MoLiNet

In this subsection, we present a novel algorithm by combining MoLiNet with distributional soft actor-critic (DSAC) (Duan et al., 2021, 2025), resulting in the MoLiNet-DSAC algorithm. It is important to clarify that DSAC here refers to its latest iteration, DSAC v2, as detailed in Duan et al. (2025). MoLiNet is designed as a general-purpose module that can be seamlessly integrated with various mainstream RL algorithms without requiring structural modifications. The architecture is inherently plug-and-play for stochastic policy-based algorithms such as SAC (Haarnoja et al., 2018), TD3 (Fujimoto et al., 2018), PPO (Schulman et al., 2017), and TRPO (Schulman et al., 2015). To validate the plug-and-play versatility of MoLiNet across different algorithm families, we have additionally integrated it with SAC and TD3, with comparative experimental results presented in Section 4.4. We feature DSAC as the primary demonstration due to its SOTA performance in continuous control tasks among model-free RL algorithms.

Different from traditional RL, the MoLiNet-DSAC evaluates the policy by learning a distribution of random returns, rather than only learning an expected return. The random return is defined as:

$$Z_{\pi_s}(s_t,a_t)=r(s_t,a_t)+\gamma G_{t+1}, \tag{13}$$

where G_{t+1} is the random soft return of policy π_s, which satisfies

$$G_t=\sum_{i=t}^{\infty}\gamma^{i-t}\left[r(s_i,a_i)-\alpha\log\pi_s(a_i|s_i)\right], \tag{14}$$

where α is the temperature coefficient.

The function $\mathcal{Z}_{\pi_s}(\cdot|s_t,a_t)$ is defined as a mapping from (s_t,a_t) to a distribution over the soft state-action return $Z_{\pi_s}(s_t,a_t)$. This mapping is referred to as soft state-action return distribution or simply value distribution. The self-consistent condition for value distribution is

$$\mathcal{T}_{\pi_s}Z_{\pi_s}(s_t,a_t)\overset{D}{=}r(s_t,a_t)+\gamma\left(Z_{\pi_s}(s_{t+1},a_{t+1})-\alpha\log\pi_s(a_t|s_t;\omega,\varphi)\right), \tag{15}$$

where $\mathcal{T}_{\pi_s}$ is the distributional Bellman operator, and $A\overset{D}{=}B$ indicates that two random variables, A and B, have identical probability laws.

The value distribution is parameterized as $\mathcal{Z}_{\pi_s}(\cdot|s_t,a_t;\theta)$, where θ are network parameters. Here, the value distribution is modeled by a Gaussian distribution, with the mean and standard deviation as its outputs. Similar to most RL methods, this algorithm follows a cycle of policy evaluation (critic update) and policy improvement (actor update).

3.2.1. Policy evaluation

The smooth critic is updated by minimizing

$$J_{\mathcal{Z}}(\theta)=\mathbb{E}\left[D_{\mathrm{KL}}\left(\mathcal{T}_{\pi_s'}\mathcal{Z}_{\pi_s'}(\cdot|s_t,a_t;\theta),\mathcal{Z}_{\pi_s}(\cdot|s_t,a_t;\theta)\right)\right], \tag{16}$$

where π_s' is the target policy and D_{KL} is the Kullback-Leibler divergence, which measures the distance between two distributions. Given that $\mathcal{T}_{\pi_s}\mathcal{Z}_{\pi_s'}(\cdot|s_t,a_t;\theta)$ is unknown, we can estimate (16) using

$$J_{\mathcal{Z}}(\theta)=-\mathbb{E}\left[\log\mathcal{P}\left(y_z|\mathcal{Z}_{\pi_s}(\cdot|s_t,a_t;\theta)\right)\right], \tag{17}$$

where y_z is the target value for random return:

$$y_z=r(s_t,a_t)+\gamma\left(Z_{\pi_s'}(s_{t+1},a_{t+1})\right)-\alpha\log\pi_s'(a_{t+1}|s_{t+1};\omega,\varphi)). \tag{18}$$

3.2.2. Policy improvement

The Lipschitz constant K will directly affect the control performance and the action smoothness. To promote a smaller K, we incorporate an L2 regularization term into the policy objective function as follows:

$$J_{\pi_s}(\omega,\varphi)=\mathbb{E}\left[Q_{\pi_s}(s_t,a_t;\theta)-\alpha\log\left(\pi_s(a_t|s_t;\omega)\right)-\lambda\|K(s_t;\varphi)\|^2\right], \tag{19}$$

where λ is the balance weight and Q_{π_s} satisfies

$$Q_{\pi_s}(s_t,a_t;\theta)=\mathbb{E}\left[Z_{\pi_s}(s_t,a_t;\theta)\right]. \tag{20}$$

Same as Haarnoja et al. (2018), the temperature coefficient α is updated using

$$\alpha\leftarrow\alpha-\beta_\alpha\mathbb{E}\left[-\log\pi_s(a_t|s_t;\omega,\varphi)-\overline{\mathcal{H}}\right], \tag{21}$$

where $\overline{\mathcal{H}}$ stands for the target entropy and β_α is the learning rate

The parameters updating process of smooth DSAC with MoLiNet is shown in Algorithm 1.

Algorithm 1 MoLiNet-DSAC.

Input: θ, ω, φ, α, $\beta_{\mathcal{Z}}$, β_π, β_k, β_α, τ, λ
Initialize target networks: $\bar{\theta}\leftarrow\theta$, $\bar{\omega}\leftarrow\omega$, $\bar{\varphi}\leftarrow\varphi$
for each iteration **do**
  for each sampling step **do**
    Calculate action $a_t\sim\pi_s(\cdot|s_t;\omega,\varphi)$
    Get reward r and new state s_{t+1}
    Store sample (s_t,a_t,r,s_{t+1}) in buffer $\mathcal{B}$
  end for
  for each update step **do**
    Sample data from $\mathcal{B}$
    Update value using $\theta\leftarrow\theta-\beta_{\mathcal{Z}}\nabla_\theta J_{\mathcal{Z}}(\theta)$
    Update policy using $\omega\leftarrow\omega+\beta_\pi\nabla_\omega J_{\pi_s}(\omega,\varphi)$
    Update constant using $\varphi\leftarrow\varphi+\beta_k\nabla_\varphi J_{\pi_s}(\omega,\varphi)$
    Update temperature using $\alpha\leftarrow\alpha-\beta_\alpha\nabla_\alpha J(\alpha)$
    Update target networks using
      $\bar{\theta}\leftarrow\tau\theta+(1-\tau)\bar{\theta}$, $\bar{\omega}\leftarrow\tau\omega+(1-\tau)\bar{\omega}$,
    $\bar{\varphi}\leftarrow\tau\varphi+(1-\tau)\bar{\varphi}$
  end for
end for

4. Simulation validation

In this section, the effectiveness of the MoLiNet-DSAC algorithm is assessed through its application in a trajectory-tracking task.

4.1. Task description

The motion of the vehicle is simulated using the dynamics model proposed by Ge et al. (2021). The state of the vehicle is represented by its position (x, y), heading angle φ, longitudinal and lateral velocities (u, v), and yaw rate ω. The control inputs include the longitudinal acceleration a_x and the front wheel steering angle δ. The actuator is bounded such that $a_x \in [-3, 3]$ m/s^2 and $\delta \in [-0.52, 0.52]$ rad.

We input the information of N reference path points in front of the vehicle into the NN as feed-forward data, as depicted in Fig. 2. Where, each reference point contains the position, heading angle, and velocity information of the reference path: $s_t^e = [x_t^e, y_t^e, \varphi_t^e, v_t^e]^T$.

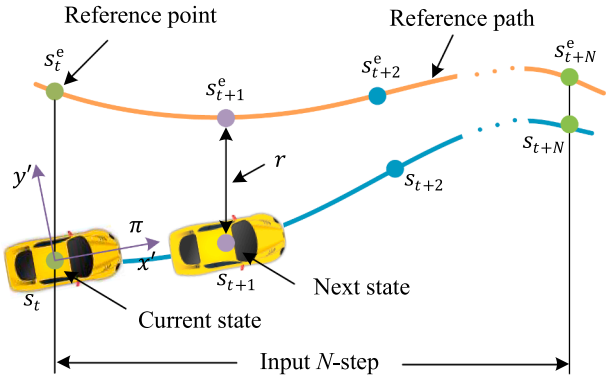


Fig. 2. Trajectory tracking diagram.

To simulate the disturbances encountered in real-world applications, we deliberately introduced varied levels of random noise to the vehicle state during testing. These noises follow a Gaussian distribution with zero mean and a standard deviation proportional to a scalar factor $i \in \{0, 1, 2, 3, 4, 5\}$. Specifically, the standard deviations are set as follows: $0.01i$ for x, v; and $0.017i$ for φ and ω. All sampled noises are directly added to the original network inputs. Moreover, we assume that the noise only affects the ego vehicle's state, while the reference path remains noise-free. Each group of experiments is conducted 100 times with randomized initial positions.

4.2. Algorithm details

To assess the performance of our proposed method in enhancing action smoothness and improving trajectory tracking accuracy, we conducted a comparative analysis with several mainstream algorithms, including DSAC (specifically, DSAC v2), SAC, TD3, and MPC. Additionally, we developed the LipsNet-DSAC algorithm, integrating DSAC with the LipsNet structure. All these algorithms are available in GOPS (Wang et al., 2023), an open-source RL platform built using PyTorch. The actor, critic, and Lipschitz constant function are represented by MLPs. The actor and critic networks each comprise two hidden layers with ReLU activation functions and 256 units per layer. The constant network features one hidden layer with ReLU activations and 128 units, while its output layer employs Softplus as the activation function. Adam serves as the optimizer for these networks. The hyperparameters specific to the vehicle trajectory tracking task are detailed in Table 1. It is important to note that all algorithm comparisons were carried out under identical experimental setups for consistency.

Table 1
Algorithm hyperparameters.

Parameter	Symbol	Value
Critic learning rate	β_Z	1e−3
Actor learning rate	β_π	1e−3
Constant learning rate	β_k	1e−4
Entropy learning rate	β_α	1e−4
Positive constant	ϵ	1e−3
Target entropy	$\bar{\mathcal{H}}$	2
Weight factor	λ	0.1
Initial value	K	[1, 1]
Prediction horizon	N	20

In the vehicle trajectory tracking experiment, sine curves were chosen as the expected trajectory, with the desired speed set to 8 m/s. Incorporating additional regularization on the control commands, the reward function is defined as

$$\begin{aligned} r_t = &-0.2\left(x_t - x_t^e\right)^2 - 0.3\left(y_t - y_t^e\right)^2 \\ &- 0.2\left(\varphi_t - \varphi_t^e\right)^2 - 0.3\left(u_t - u_t^e\right)^2 \\ &- 0.1v^2 - 0.1\omega^2 - 0.05a_x^2 - 0.05\delta^2, \end{aligned} \tag{22}$$

where $x_t^e, y_t^e, \varphi_t^e, v_t^e$ represent reference states at time t.

4.3. Results

To compare the performance of different algorithms, we computed the action fluctuation and tracking errors of MoLiNet-DSAC and other algorithms at various noise amplitudes. Each evaluation was repeated 100 times with distinct but consistent initial states, as shown in Fig. 3 and summarized in Table 2. Under the same noise conditions, the action fluctuations of TD3 and SAC algorithms were significantly higher than those of algorithms such as DSAC, accompanied by lower cumulative returns. While LipsNet-DSAC exhibits lower action fluctuation, the inherent characteristics of the LipsNet network make it challenging to accommodate the control characteristics of different action components, resulting in larger tracking errors. In contrast, our MoLiNet-DSAC algorithm demonstrated excellent action smoothing capabilities across different noise amplitudes. The growth in action fluctuation was notably lower than that observed in traditional RL algorithms and MPC algorithms. Moreover, under the influence of six different amplitudes of noise, MoLiNet-DSAC maintained stable tracking accuracy, with an average position error of less than 5cm. As the MoLiNet-DSAC fluctuation further decreased, the position tracking accuracy achieved a 2.5 times improvement compared to LipsNet-DSAC. Compared to the DSAC algorithm, the smoothness of action has increased by factors of 1.1, 8.0, 8.2, 8.2, 8.3, and 8.5 across six noise amplitudes. These results indicate that MoLiNet-DSAC retains high trajectory tracking accuracy and smooth action capabilities even in the presence of noise.

Fig. 4 displays the action and state curves under the influence of two different noise amplitudes. Due to noise interference, the control actions of all three algorithms exhibit some degree of fluctuation. However, the action curve of MoLiNet-DSAC is noticeably smoother in comparison to DSAC and MPC. This further illustrates that MoLiNet-DSAC, even amidst noisy conditions, effectively minimizes motion fluctuations while maintaining superior trajectory tracking capabilities. This underscores the ability of MoLiNet-DSAC to achieve a collaborative optimization between control precision and action smoothness.

Finally, we computed the average single-step inference time for MPC (utilizing the Ipopt solver), DSAC, LipsNet-DSAC, and MoLiNet-DSAC under a noise amplitude of 0, with the results summarized in Table 3. All tests were conducted on a Windows platform powered by an Intel 12700KF processor. With a prediction horizon set at 20 steps, MPC requires an average single-step solution time of approximately 33 ms with a notably high standard deviation of 83.75 ms, reflecting the in-

Table 2
Tracking error and standard deviation under various noise amplitudes.

Noise	State error	Algorithms			
		M-DSAC	L-DSAC	DSAC	MPC
0	Δy	4.44 ± 0.06	11.26 ± 0.03	4.50 ± 0.01	0.29 ± 0.07
	Δv	4.22 ± 0.18	4.34 ± 0.11	4.69 ± 0.05	2.59 ± 0.51
1	Δy	4.43 ± 0.06	11.27 ± 0.05	4.51 ± 0.05	0.27 ± 0.07
	Δv	4.20 ± 0.15	4.34 ± 0.09	4.80 ± 0.08	2.63 ± 0.66
2	Δy	4.45 ± 0.07	11.26 ± 0.08	4.65 ± 0.10	0.26 ± 0.08
	Δv	4.22 ± 0.17	4.34 ± 0.10	5.08 ± 0.15	2.71 ± 0.59
3	Δy	4.41 ± 0.12	11.30 ± 0.11	4.66 ± 0.19	0.27 ± 0.08
	Δv	4.19 ± 0.18	4.31 ± 0.11	5.44 ± 0.20	2.43 ± 0.49
4	Δy	4.36 ± 0.14	11.30 ± 0.13	4.68 ± 0.25	0.26 ± 0.07
	Δv	4.22 ± 0.21	4.33 ± 0.12	5.79 ± 0.28	2.47 ± 0.48
5	Δy	4.32 ± 0.19	11.36 ± 0.18	4.65 ± 0.26	0.25 ± 0.04
	Δv	4.20 ± 0.19	4.28 ± 0.09	6.21 ± 0.22	2.78 ± 0.46

M-DSAC: MoLiNet-DSAC; L-DSAC: LipsNet-DSAC. Unit: Δy in cm, Δv in cm/s.

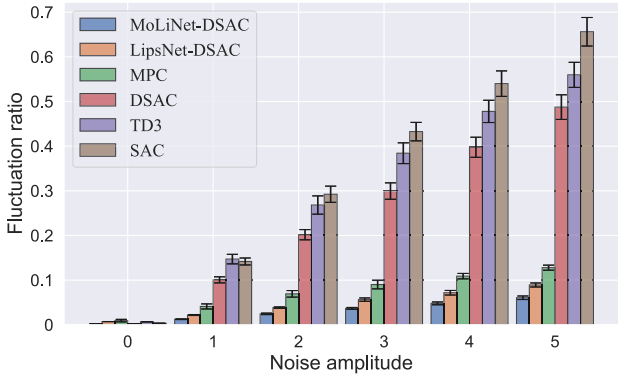


Fig. 3. Trends in action fluctuation ratio. Error bars represent the 95% confidence intervals over 100 runs with random initial conditions.

Table 3
Average single-step solution time.

Algorithms	Mean/ms	Std/ms
DSAC	0.20	0.01
LipsNet-DSAC	0.66	0.06
MoLiNet-DSAC	0.70	0.09
MPC	33.00	83.75

herent variability of online optimization under dynamic resource allocation. In contrast, all three RL-based methods achieve significantly faster inference: DSAC attains the lowest latency at 0.20 ± 0.01 ms, while LipsNet-DSAC and MoLiNet-DSAC incur a modest additional overhead due to the Lipschitz normalization module, achieving 0.66 ± 0.06 ms and 0.70 ± 0.09 ms, respectively. Notably, MoLiNet-DSAC is approximately 47 times faster than MPC, with a substantially more stable solution time, demonstrating that the introduction of the MoLiNet module imposes negligible computational cost relative to baseline DSAC while achieving a dramatic reduction in latency compared to optimization-based methods. This confirms that MoLiNet-DSAC is well-suited for real-time deployment in resource-constrained or high-frequency control scenarios.

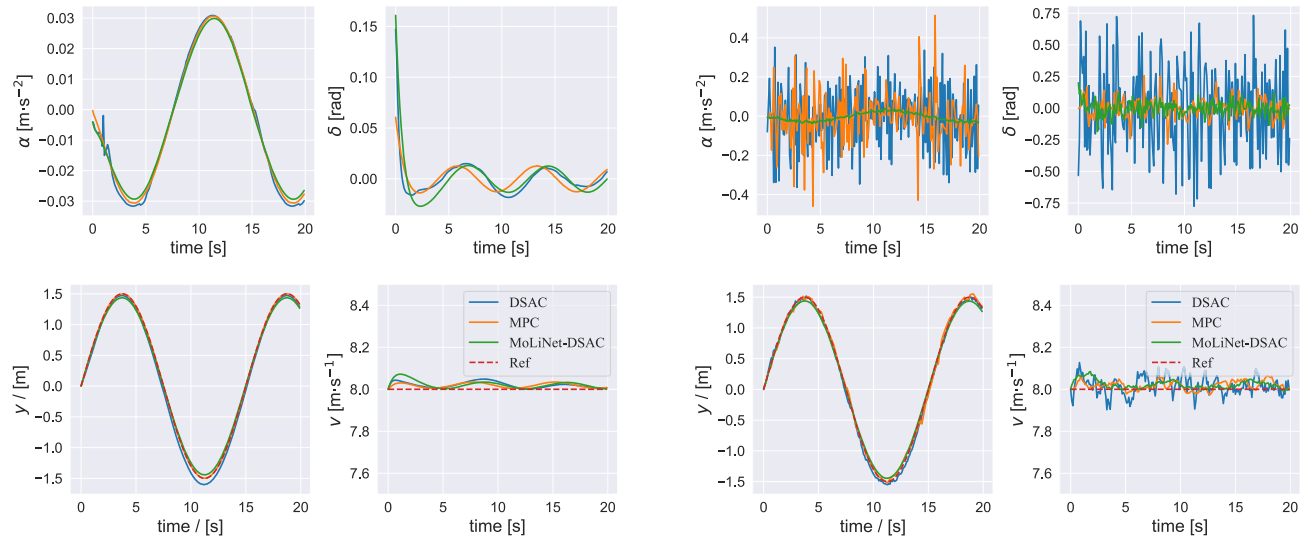
4.4. Parameter sensitivity analysis

We first validate the plug-and-play property and training stability of MoLiNet by integrating it with two additional representative RL algorithms: SAC and TD3. Fig. 5 shows the training curves obtained using five different random seeds for each algorithm. Incorporating MoLiNet does not introduce training instability, as the learning curves remain smooth and exhibit comparable or reduced variance in episode returns. The volatility analysis (rightmost panel) quantifies the training stability improvements, where MoLiNet-augmented variants demonstrate lower fluctuation magnitudes during training. Moreover, the introduction of the Lipschitz constraint does not impose noticeable optimization difficulties, with convergence speeds similar to or faster than baseline algorithms. Both MoLiNet-augmented variants achieve improved final performance with higher episode returns.

Beyond the Gaussian noise setting adopted in the main experiments, we further investigate the robustness of MoLiNet under a uniform noise distribution to evaluate the distribution-agnostic property of the Lipschitz constraint. Fig. 6a reports the performance comparison under Gaussian and uniform noise at noise level 5 with $\lambda = 0.1$. The uniform noise is independently sampled for each state dimension from $\mathcal{U}[-0.05, 0.05]$, which matches the variance range of the Gaussian noise. The results show that MoLiNet-DSAC achieves a fluctuation ratio of 0.011 under uniform noise compared to 0.015 under Gaussian noise, with negligible performance degradation (average return: -0.33 vs -0.32). The results indicate that MoLiNet-DSAC achieves comparable action smoothness and control performance under both noise distributions, suggesting that the imposed Lipschitz bound provides robustness that is not tied to a specific noise model.
.

We further analyze the sensitivity of MoLiNet with respect to the weighting factor λ by evaluating the action fluctuation ratio and average episode return over a wider range of values, $\lambda \in [0.01, 1.0]$, as summarized in Fig. 6b. As λ increases, the learned policy becomes smoother, while the overall control performance gradually degrades. Specifically, increasing λ from 0.01 to 1.0 reduces the fluctuation ratio from 0.034 to 0.008, representing a 76% improvement in action smoothness. The experimental results show that $\lambda \approx 0.1$ yields a favorable trade-off between action smoothness and task performance. Overall, the results indicate that MoLiNet exhibits limited sensitivity to λ within a reasonable range, facilitating practical parameter selection.

Finally, we compare different regularization forms used to constrain the Lipschitz constants, specifically L2 and L1 regularization. Fig. 6c presents the results obtained under identical experimental settings at noise amplitude 5. While L1 regularization achieves a slightly lower fluctuation ratio (0.011 vs 0.015), it incurs a substantial performance penalty with an average return of -0.75 compared to -0.32 for L2 regularization. While both regularization strategies are effective in reducing action fluctuations, L2 regularization leads to more stable convergence behavior and a smoother trade-off between control performance and action smoothness. In contrast, L1 regularization tends to induce abrupt changes in the policy due to its sparsity-promoting nature, which may result in overly restricted or insensitive action dimensions. These observations suggest that L2 regularization is more suitable for the considered multidimensional continuous control task.



(a) Noise = 0 **(b)** Noise = 5

Fig. 4. Action and state curves under different noise levels. Each subfigure shows the action (top) and corresponding state (bottom) trajectories for the same noise setting.

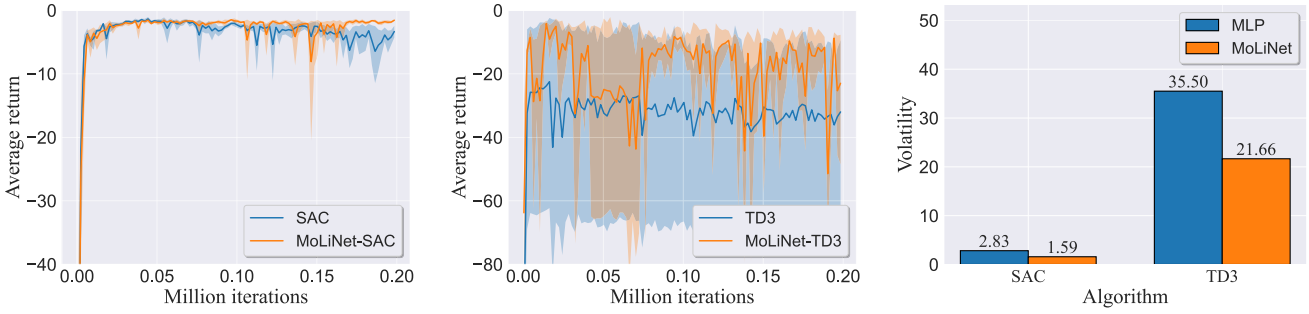
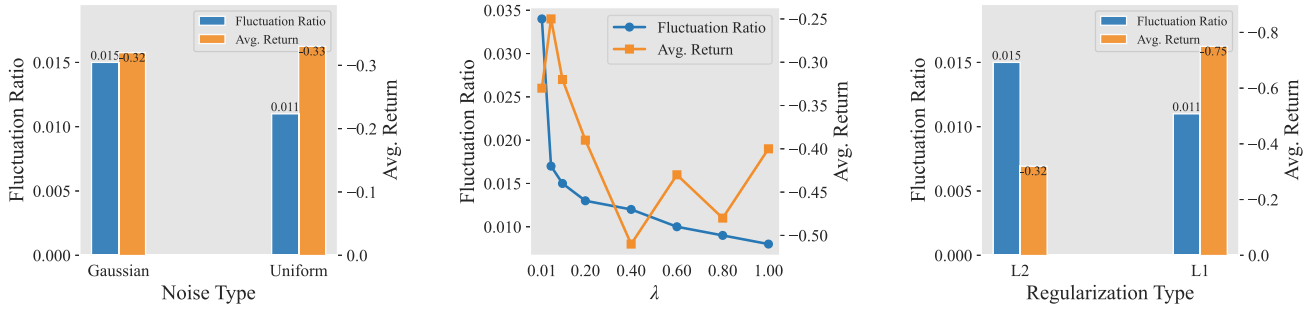


Fig. 5. Training convergence and volatility analysis under multiple random seeds. Left: SAC vs. MoLiNet-SAC. Middle: TD3 vs. MoLiNet-TD3. Right: Volatility comparison. Each convergence curve represents the mean episode return over 5 random seeds with shaded regions indicating standard deviation.



(a) Noise Type Comparison **(b)** Weight Factor λ Comparison **(c)** Regularization Comparison

Fig. 6. Performance comparison under different configurations. (a) Fluctuation ratio and average return for Gaussian and Uniform noise types. (b) Performance metrics across different weight factors λ. (c) Comparison between L2 and L1 regularization methods.

5. Experimental validation

In this section, we deploy the policy learned by MoLiNet-DSAC to control a wheeled vehicle in performing trajectory tracking and collision avoidance tasks.

5.1. Task description

In this experiment, we used the Geek+ M200 logistics robot, which employs differential steering. The robot's key parameters are summarized in Table 4. The state variables include the position (longitudinal

Table 4
Vehicle parameters.

Description	Symbol	Value
Rotation diameter	d	774 mm
Wheel track	d_w	423 mm
Tyre diameter	D	150 mm
Mass	m	68 kg
Size	$l_x \times l_y \times l_z$	$760 \times 520 \times 210$mm

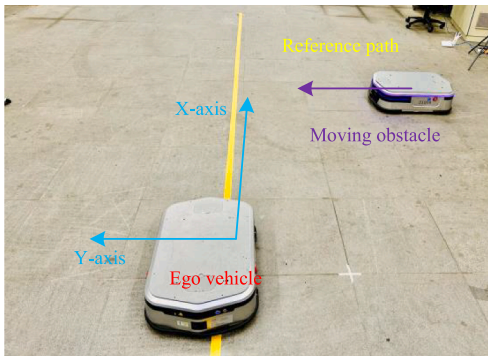


Fig. 7. Vehicle trajectory tracking and collision avoidance task.

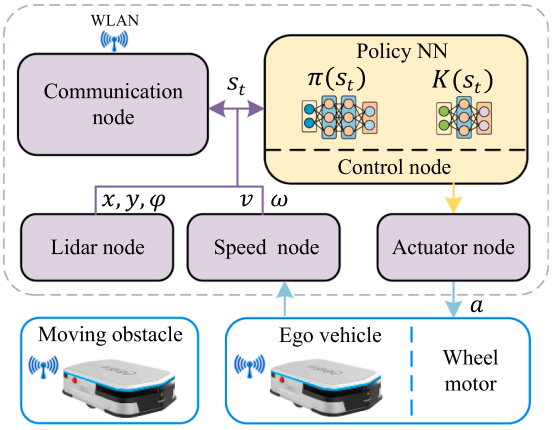


Fig. 8. Experiment framework.

x and lateral y), heading angle φ, longitudinal velocity v, and yaw rate ω. The control inputs consist of velocity increments δv and angular velocity increments $\delta\omega$, which influence the robot's movement. The control frequency was set to $f = 10$ Hz, with velocity increments limited to $|\delta v| \le 1.8/f$ and angular velocity increments restricted to $|\delta\omega| \le 0.8/f$.

The ego vehicle's initial position is randomized, with the desired trajectory along the x-axis in Fig. 7 and a reference speed of 0.3 m/s. The moving obstacle follows a pre-defined path at a constant speed and direction. The ego vehicle uses LiDAR for position and heading estimation, and sensors for current speed and angular velocity. It also receives data on the moving obstacle's position and velocity. The policy network generates a control signal based on this sensor data, which is relayed to the actuator node via ROS, creating a closed-loop control system, as shown in Fig. 8.

5.2. Experimental details

The reward of the task is defined as

$$r_t = -2.4(y_t - 0)^2 - 1.6(\varphi_t - 0)^2 - 1.0(v_t - 0.3)^2 - 0.2\omega^2 - 0.1(\delta v)^2 - 0.1(\delta w)^2 + r_c, \tag{23}$$

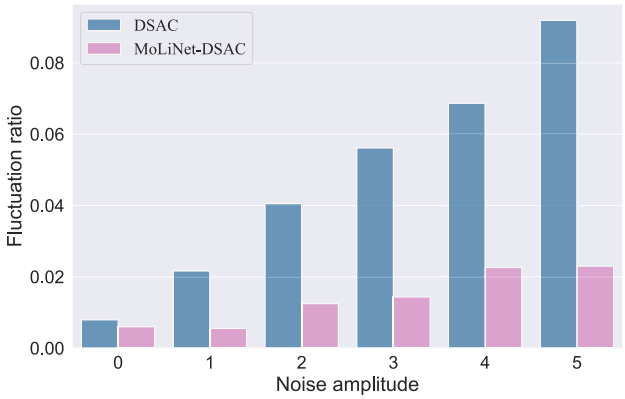


Fig. 9. Fluctuation ratio change trend.

where r_c is the collision term, and if the vehicle collides with the moving obstacle, the vehicle will receive a penalty of −500; otherwise, the term is 0.

The state representation of this task is designed as

$$s = \{v, \omega, \Delta d, \Delta\varphi, \Delta v, \Delta\bar{x}, \Delta\bar{y}, \Delta\bar{\varphi}, \bar{v}\}, \tag{24}$$

which encompasses both the vehicle state information and the moving obstacle information. Here, Δd, $\Delta\varphi$, and Δv represent the lateral deviation, heading deviation, and speed deviation, respectively. Similarly, $\Delta\bar{x}$, $\Delta\bar{y}$, and $\Delta\bar{\varphi}$ correspond to the longitudinal distance, lateral distance, and angular difference relative to the moving obstacle. The term $\bar{v}$ signifies the speed of the moving obstacle.

Mirroring the approach taken in the simulation experiment, we introduce varying amplitudes of random noise to the input of the policy neural network. These noises follow a Gaussian distribution with zero mean and a standard deviation proportional to a scalar factor $i \in \{0, 1, 2, 3, 4, 5\}$. Specifically, the standard deviation is set to $0.01i$ for the linear velocity v, relative distance Δd, relative velocity Δv, estimated position components $\Delta\bar{x}, \Delta\bar{y}$, and estimated speed $\bar{v}$; and $0.017i$ for the angular velocity ω, relative heading $\Delta\varphi$, and estimated heading $\Delta\bar{\varphi}$. The architecture of the network and the primary hyperparameters remain consistent with those described in Section 5.2. We stress that, in all experiments, the ego vehicle does not know the behavior or trajectory of the moving obstacle in advance.

5.3. Results and discussion

The policy network is trained using both MoLiNet-DSAC and DSAC algorithms based on the dynamic model, after which the trained network is implemented on the actual vehicle. A video for the results is available at https://youtu.be/5wrVQZ7P95g. We highly recommend readers to watch the video for an intuitive evaluation.

We examine action fluctuations for both the MoLiNet-DSAC and DSAC algorithms across six noise scenarios, and the results are shown in Fig. 9. Throughout the trajectory tracking process, both MoLiNet-DSAC and DSAC effectively accomplish the collision avoidance task. As the noise amplitude increases, the action fluctuation ratio of DSAC increases significantly compared with MoLiNet-DSAC, while MoLiNet-DSAC shows strong noise robustness. Specifically, at high noise amplitudes, the action fluctuation ratio for MoLiNet-DSAC is approximately one-fourth that of DSAC, indicating superior action smoothing effectiveness. Another noteworthy point is that no additional noise was introduced during training. Nevertheless, MoLiNet-DSAC still demonstrates strong noise robustness in real-world deployment. This is because the constant network can dynamically adjust the output Lipschitz constraints based on the observed variations, thereby limiting the gradient changes of the policy network, preventing small disturbances from escalating into drastic action variations.

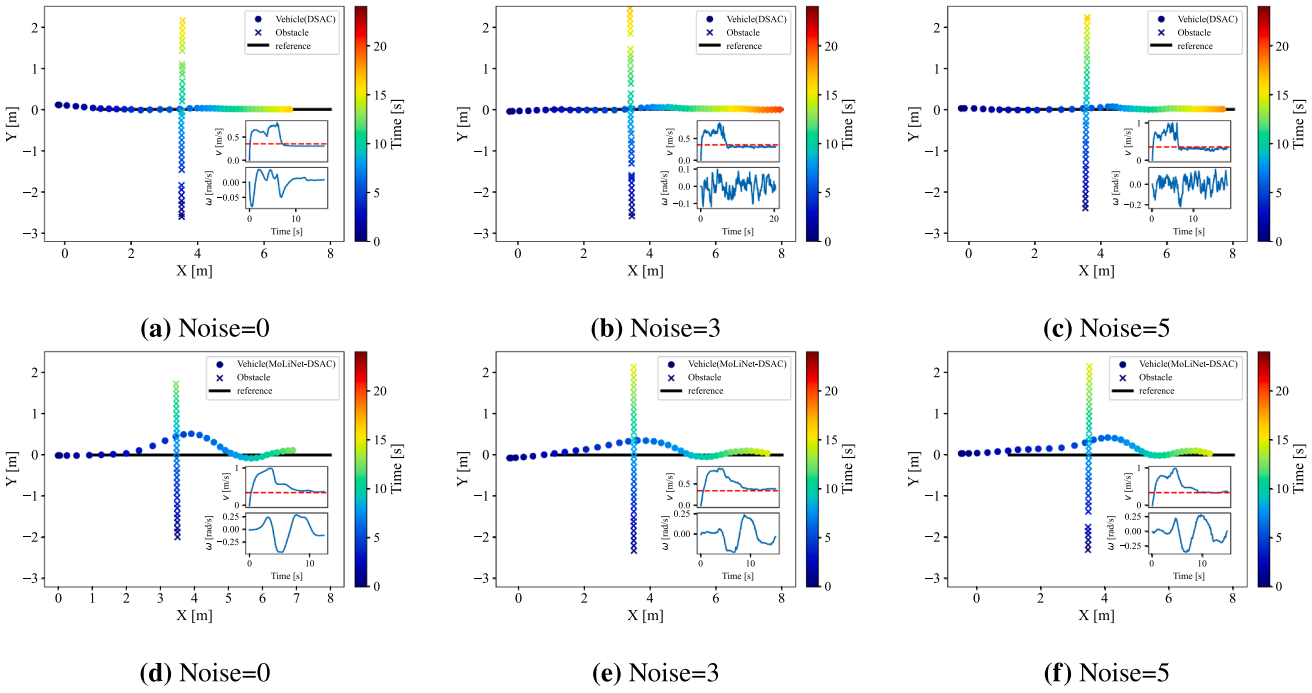


Fig. 10. Vehicle state diagram. Trajectories of ego vehicle and moving obstacle in three noise scenarios. All points are plotted at the same time intervals, so the density of points also represents the speed of the object. The sub-figure also plots the vehicle's speed and angular velocity curves. DSAC: (a)–(c), MoLiNet-DSAC: (d)–(f).

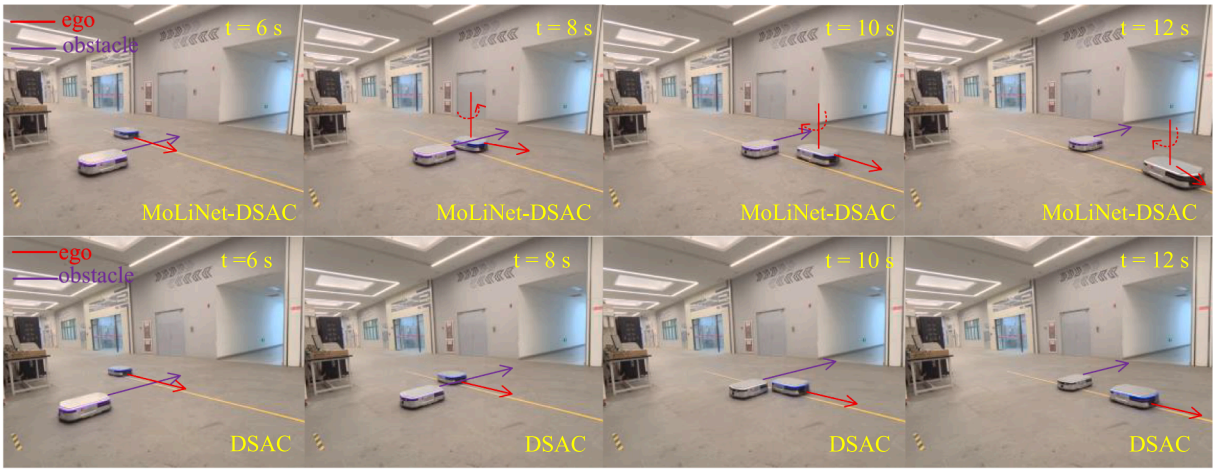


Fig. 11. Experimental snapshot. The noise amplitude is set to 5, the red arrow represents the direction of ego vehicle movement, and the purple represents the direction of obstacle movement. At t = 8s, the ego vehicle controlled by MoLiNet-DSAC began to accelerate and turn, maintaining a large distance from the obstacle. At t = 10s, the ego vehicle controlled by DSAC almost collided with the obstacle.

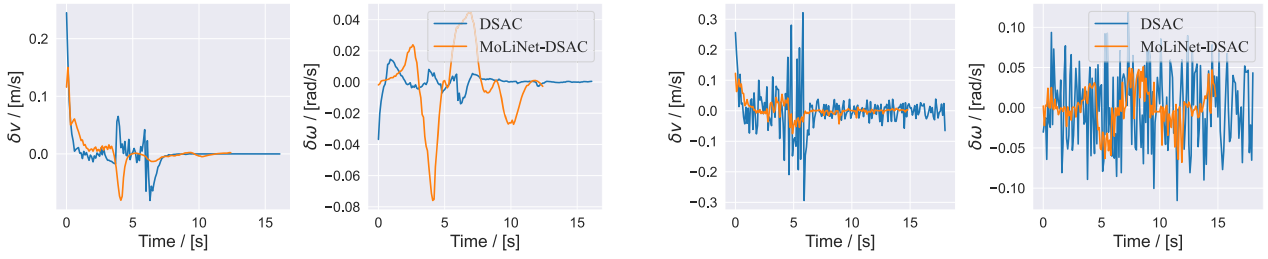
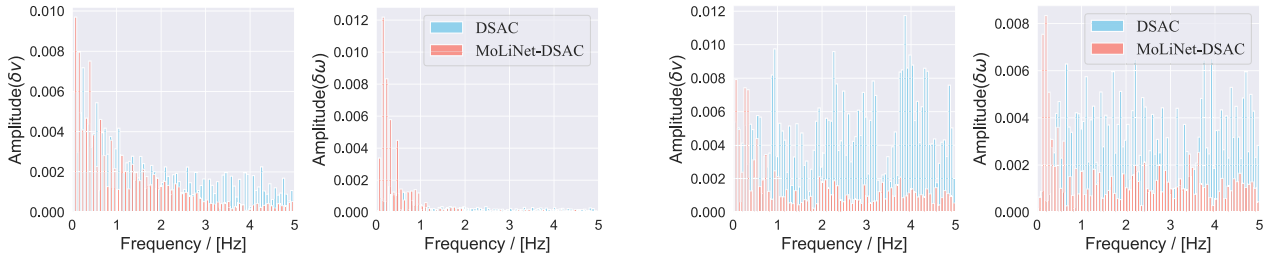


Fig. 12. Action fluctuations.



(a) Noise=0 **(b)** Noise=5

Fig. 13. Frequency composition.

Fig. 10 illustrates the control results of the policy network in three noise scenarios. When a slowly approaching moving obstacle is encountered, the DSAC algorithm proactively accelerates to avoid collision but refrains from distinct steering maneuvers. In contrast, the MoLiNet-DSAC algorithm, when faced with moving obstacles, opts for both active acceleration and steering avoidance, ensuring a larger safe distance is maintained. Since MoLiNet imposes independent Lipschitz continuity constraints on each action component, the policy network tends to simultaneously adjust both acceleration and steering in the presence of noise interference, thereby avoiding abrupt changes in individual actions and improving the overall smoothness of the control actions. Specific experimental snapshots under high noise amplitudes are presented in Fig. 11. Notably, in comparison, MoLiNet-DSAC exhibits conspicuous steering behavior, whereas DSAC nearly leads to a collision around $t = 10$s. While both algorithms achieve the goal of collision avoidance, the DSAC strategy, under the influence of high-amplitude noise, demonstrates considerable control action fluctuations. The speed and heading angle of the vehicle become unstable. In contrast, the state trajectory observed in MoLiNet-DSAC is more stable, highlighting its superior performance.

More intuitively, we list the action fluctuations in two noise scenarios in Fig. 12. In comparison, the control action curve of our method is smoother. In contrast, the action curve of DSAC shows obvious jitter, which seriously affects the driving stability of the vehicle in the experiment. In addition, we compared the action frequency composition in the two noise scenarios in the frequency domain. The results are shown in Fig. 13. Compared with DSAC, the frequency composition of our method is mainly concentrated in the low-frequency range, which shows that our method has the characteristics of low-pass filtering, while the frequency composition of the DSAC algorithm is more scattered and cannot effectively achieve smooth action in the face of noise interference. These results further illustrate that our proposed MoLiNet-DSAC method has strong noise robustness and action-smoothing capabilities.

6. Conclusion

In this study, we propose an RL policy neural network architecture with multidimensional action smoothing capabilities, which adaptively regulates individual Lipschitz constants for each element of multidimensional action vectors through a trainable parameterized network, thereby achieving smooth control actions. The network can be seamlessly integrated into most existing RL algorithms without any modification. Furthermore, to facilitate the acquisition of smooth control policies, we combine MoLiNet with distributed soft actor-critic (DSAC) and introduce an innovative RL algorithm named MoLiNet-DSAC. MoLiNet-DSAC integrates the L2 regularization term corresponding to Lipschitz constants into the RL loss function during training, aiming to achieve a dynamic balance between control performance and smoothness. The policy learned by MoLiNet-DSAC is tested on an actual wheeled vehicle for executing trajectory tracking and collision avoidance, where it demonstrates not just improved action smoothness but also a substantial increase in position tracking precision, outperforming DSAC with LipsNet by over threefold. Future work will delve into the potential of MoLiNet-DSAC across more intricate tasks and varied environments.

Credit Author Statement

L.C. and X.S. contributed equally to this work. J.D. supervised the project. All authors reviewed and approved the manuscript.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by Beijing Natural Science Foundation under Grant L257002, in part by the National Science Foundation China under Grant 62273256, and in part by the Fundamental Research Funds for the Central Universities under Grant FRF-OT-23-02.

Appendix A.

Wu et al. (2021) proposed the Gradient Normalization (GN) method to constrain a neural network $\pi : \mathbb{R}^n \to \mathbb{R}$ to be 1-Lipschitz continuous at the network level. In MoLiNet, we use different Lipschitz constants to implement independent constraints for different control actions, which is equivalent to using GN for each action to ensure that the mapping function from state to each action dimension is C-Lipschitz continuous. For the completeness of the article, our following proof process presented below is inspired by GN.

Lemma 1. *Suppose $\pi : \mathbb{R}^n \to \mathbb{R}$ is a continuously differential function. The C-Lipschitz continuity constraint*

$$\|\pi(x) - \pi(y)\| \le C\|x - y\|, \ \forall x, y \in \mathbb{R}^n. \tag{25}$$

is equivalent to

$$\|\nabla_x \pi(x)\| \le C, \forall x \in \mathbb{R}^n. \tag{26}$$

Proof. Firstly, we prove the sufficient condition.
(⇒) From the definition of Lipschitz continuity constraint (25), we know

$$\|\pi(x) - \pi(y)\| \le C\|x - y\|, \tag{27}$$

where we consider the norm of directional derivative at x along with the direction of $(y - x)$:

$$\langle \nabla_x \pi(x), \frac{y - x}{\|y - x\|} \rangle = \lim_{y \to x} \frac{\pi(y) - \pi(x)}{\|x - y\|} \le C. \tag{28}$$

Here, the $\langle \cdot, \cdot \rangle$ is the inner product. Since the norm of gradient is the maximum norm of directional derivative, then

$$\|\nabla_x \pi(x)\| \le C. \tag{29}$$

Then we prove the necessary condition.
(⇐) We assume that π is continuous and differentiable. Let $f(t) = \pi(x + t(y - x))$ and $f'(t) = \nabla\pi(x + t(y - x)) \cdot (y - x)$, where $t \in \mathbb{R}$, $y \in \mathbb{R}^n$. According to the Newton-Leibniz formula, we can get

$$f(1) - f(0) = \int_0^1 f'(t)\, dt. \tag{30}$$

By taking the 2-norm on both sides, one has

$$\begin{aligned} \|\pi(y) - \pi(x)\| &= \left\| \int_0^1 (\nabla\pi(x + t(y - x))\, dt)(y - x) \right\| \\ &\le \left(\int_0^1 \|\nabla\pi(x + t(y - x))\|\, dt \right) \|y - x\| \\ &\le \left(\int_0^1 C\, dt \right) \|y - x\| \\ &= C\|y - x\|. \end{aligned} \tag{31}$$

□

Now we are ready to present the proof of Theorem 1.

Proof. By definition, the gradient norm of $\pi_s(\cdot)[i]$ is

$$\begin{aligned} &\|\nabla\pi_s(\cdot)[i]\| \\ &= K[i] \cdot \left\| \nabla \left(\frac{\pi[i]}{\|\nabla\pi[i]\| + \epsilon} \right) \right\| \\ &= K[i] \cdot \left\| \frac{\nabla\pi[i] \cdot (\|\nabla\pi[i]\| + \epsilon) - \pi[i] \cdot \nabla(\|\nabla\pi[i]\| + \epsilon)}{(\|\nabla\pi[i]\| + \epsilon)^2} \right\|. \end{aligned} \tag{32}$$

Since the network π contains only piecewise linear activation functions, the gradient vector $\nabla\pi[i]$ is a constant matrix. So, $\nabla(\|\nabla\pi[i]\| + \epsilon)$ is a zero matrix. The Eq. (32) can be simplified to

$$\|\nabla\pi_s(\cdot)[i]\| = K[i] \cdot \left\| \frac{\nabla\pi[i]}{\|\nabla\pi[i]\| + \epsilon} \right\| \le C_i. \tag{33}$$

By Lemma 1, we complete the proof of Theorem 1. □

References

Chen, C., Tang, H., Hao, J., Liu, W., & Meng, Z. (2021). Addressing action oscillations through learning policy inertia. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 7020–7027). (*vol. 35*).

Duan, J., Cao, W., Zheng, Y., & Zhao, L. (2024). On the optimization landscape of dynamic output feedback linear quadratic control. *IEEE Trans. Automatic Control*, *69*(2), 920–935.

Duan, J., Guan, Y., Li, S. E., Ren, Y., Sun, Q., & Cheng, B. (2021). Distributional soft actor-critic: Off-policy reinforcement learning for addressing value estimation errors. *IEEE Trans. Neural Netw. Learn. Syst.*, *33*(11), 6584–6598.

Duan, J., Ren, Y., Zhang, F., Li, J., Li, S. E., Guan, Y., & Li, K. (2024). Encoding distributional soft actor-critic for autonomous driving in multi-lane scenarios [research frontier]. *IEEE Comput. Intell. Magaz.*, *19*(2), 96–112.

Duan, J., Wang, W., Xiao, L., Gao, J., Li, S. E., Liu, C., Zhang, Y.-Q., Cheng, B., & Li, K. (2025). Distributional soft actor-critic with three refinements. *IEEE Trans. Pattern Analy. Mach. Intell.*, *47*(5), 3935–3946.

Elhaki, O., Shojaei, K., & Mehrmohammadi, P. (2022). Reinforcement learning-based saturated adaptive robust neural-network control of underactuated autonomous underwater vehicles. *Expert Syst. Appl.*, *197*, 116714. https://www.sciencedirect.com/science/article/pii/S0957417422001907. https://doi.org/10.1016/j.eswa.2022.116714

Fujimoto, S., Hoof, H., & Meger, D. (2018). Addressing function approximation error in actor-critic methods. In *International conference on machine learning* (pp. 1587–1596). PMLR.

Ge, Q., Sun, Q., Li, S. E., Zheng, S., Wu, W., & Chen, X. (2021). Numerically stable dynamic bicycle model for discrete-time control. In *2021 IEEE intelligent vehicles symposium workshops (IV workshops)* (pp. 128–134). IEEE.

Grüne, L., Pannek, J., Grüne, L., & Pannek, J. (2017). Nonlinear Model Predictive Control. Springer.

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning* (pp. 1861–1870). PMLR.

He, X., Yang, H., Hu, Z., & Lv, C. (2022). Robust lane change decision making for autonomous vehicles: an observation adversarial reinforcement learning approach. *IEEE Trans. Intell. Veh.*, *8*(1), 184–193.

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, *2*(5), 359–366.

Kobayashi, T. (2022). L2c2: locally lipschitz continuous constraint towards stable and smooth reinforcement learning. In *2022 IEEE/RSJ international conference on intelligent robots and systems (IROS)* (pp. 4032–4039). IEEE.

Li, M., Wang, B., Wang, S., & Ke, Z. (2025). Serial distributed reinforcement learning for enhanced multi-objective platoon control in curved road coordinates. *Expert Syst. Appl.*, *269*, 126493. https://www.sciencedirect.com/science/article/pii/S0957417425001150. https://doi.org/10.1016/j.eswa.2025.126493

Li, S. E. (2023). Reinforcement Learning for Sequential Decision and Optimal Control. Springer Verlag, Singapore.

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2015). Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971.

Lu, J., Li, L., & Wang, F.-Y. (2024). Event-triggered parallel control using deep reinforcement learning with application to comfortable autonomous driving. *IEEE Trans. Intell. Veh.*, *9*(3), 4470–4479.

Luo, Z., Zhou, J., & Wen, G. (2022). Deep reinforcement learning based tracking control of unmanned vehicle with safety guarantee. In *2022 13th Asian control conference (ASCC)* (pp. 1893–1898). IEEE.

Ma, J., Cheng, Z., Zhang, X., Tomizuka, M., & Lee, T. H. (2022). Alternating direction method of multipliers for constrained iterative LQR in autonomous driving. *IEEE Trans. Intell. Transp. Syst.*, *23*(12), 23031–23042.

Mysore, S., Mabsout, B., Mancuso, R., & Saenko, K. (2021). Regularizing action policies for smooth control with reinforcement learning. In *2021 IEEE international conference on robotics and automation (ICRA)* (pp. 1810–1816). IEEE.

Peng, B., Duan, J., Chen, J., Li, S. E., Xie, G., Zhang, C., Guan, Y., Mu, Y., & Sun, E. (2022). Model-based chance-constrained reinforcement learning via separated proportional-integral lagrangian. *IEEE Trans. Neural Netw. Learn. Syst.*, *35*(1), 466–478.

Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). Trust region policy optimization. In *International conference on machine learning* (pp. 1889–1897). PMLR.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.

Shen, Q., Li, Y., Jiang, H., Wang, Z., & Zhao, T. (2020). Deep reinforcement learning with robust and smooth policy. In *International conference on machine learning* (pp. 8707–8718). PMLR.

Song, X., Duan, J., Wang, W., Li, S. E., Chen, C., Cheng, B., Zhang, B., Wei, J., & Wang, X. S. (2023). LipsNet: a smooth and robust neural network with adaptive lipschitz constant for high accuracy optimal control. In *Proceedings of the 40th international conference on machine learning* (pp. 32253–32272). PMLR (*vol. 202*). Proceedings of Machine Learning Research.

Takase, R., Yoshikawa, N., Mariyama, T., & Tsuchiya, T. (2022). Stability-certified reinforcement learning control via spectral normalization. *Mach. Learn. Appl.*, *10*, 100409.

Udekwe, D., Ajayi, O.-o., Ubadike, O., Ter, K., & Okafor, E. (2024). Comparing actor-critic deep reinforcement learning controllers for enhanced performance on a ball-and-plate system. *Expert Syst. Appl.*, *245*, 123055. https://www.sciencedirect.com/science/article/pii/S0957417423035571. https://doi.org/10.1016/j.eswa.2023.123055

Wang, W., Zhang, Y., Gao, J., Jiang, Y., Yang, Y., Zheng, Z., Zou, W., Li, J., Zhang, C., Cao, W. et al. (2023). Gops: a general optimal control problem solver for autonomous driving and industrial control applications. *Commun. Transp. Res.*, *3*, 100096.

Wang, Z., Huang, H., Tang, J., & Hu, L. (2024). A deep reinforcement learning-based approach for autonomous lane-changing velocity control in mixed flow of vehicle group level. *Expert Syst. Appl.*, *238*, 122158. https://www.sciencedirect.com/science/article/pii/S095741742302660X. https://doi.org/10.1016/j.eswa.2023.122158

Wu, S., Wu, J., Cao, X., Zhang, F., Yu, G., Zhao, J., Qu, Y., Ma, F., & Duan, J. (2026). Transformer-based explicit model predictive control with variable prediction horizon. *Eng. Appl. Artif. Intell.*, *172*, 114345. https://www.sciencedirect.com/science/article/pii/S0952197626006263. https://doi.org/10.1016/j.engappai.2026.114345

Wu, Y.-L., Shuai, H.-H., Tam, Z.-R., & Chiu, H.-Y. (2021). Gradient normalization for generative adversarial networks. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 6373–6382).

Yu, H., Xu, W., & Zhang, H. (2021). Taac: temporally abstract actor-critic for continuous control. *Adv. Neural Inf. Process. Syst.*, *34*, 29021–29033.



Liangfa Chen received the B S degree from the School of Technology, Beijing Forestry University, in 2022, and the M S degree in Mechanical Engineering from the University of Science and Technology Beijing in 2025. He is currently an Research Engineer at SunRising AI Ltd. His research interests include reinforcement learning, optimal control, and decision-making for autonomous driving systems.



Xujie Song received the M S degree in Mechanical Engineering from Tsinghua University in 2024. He is currently pursuing a PhD degree in Civil Engineering at the University of Michigan, USA. His research interests include reinforcement learning, autonomous driving, and robotics.



Junjie Zhao is currently pursuing the M S degree in Mechanical Engineering at the University of Science and Technology Beijing. His research interests include reinforcement learning, optimal control, and autonomous driving decision-making.



Wenxuan Wang received the B S degree in Vehicle Engineering from Beijing Institute of Technology in 2018. He is currently with the State Key Laboratory of Automotive Safety and Energy, School of Vehicle and Mobility, Tsinghua University. His research interests include decision-making and control of autonomous vehicles and reinforcement learning.



Yinuo Wang received the B.S. degree in Automation from the University of Science and Technology Beijing in 2023. He is currently pursuing the M.S. degree in Vehicle Engineering at Tsinghua University. His research interests include deep learning, reinforcement learning, optimal control, diffusion models, and autonomous driving.



Liming Xiao received the B S degree in Vehicle Engineering from the University of Science and Technology Beijing in 2023. He is currently pursuing the M S degree in Mechanical Engineering at the University of Science and Technology Beijing. His research interests include reinforcement learning, optimal control, and autonomous driving decision-making.



Shengbo Eben Li (Senior Member, IEEE) received the M S and PhD degrees from Tsinghua University in 2006 and 2009, respectively. He worked at Stanford University, the University of Michigan, and the University of California, Berkeley. He is currently a tenured professor at Tsinghua University. His research interests include intelligent vehicles, driver assistance systems, reinforcement learning, distributed control, and optimal control. He serves as an Associate Editor of IEEE ITSM and IEEE Transactions on ITS.



Fei Ma received the M S and PhD degrees in Mechanical Engineering from the University of Science and Technology Beijing in 1993 and 2005, respectively. He is currently a Tenured Professor and Dean of the School of Mechanical Engineering, University of Science and Technology Beijing. His research interests include vehicle dynamics, condition monitoring, and fault diagnosis of machinery.



Jingliang Duan received the PhD degree from the School of Vehicle and Mobility, Tsinghua University, in 2021. He was a visiting student researcher at the University of California, Berkeley, in 2019, and a research fellow at the National University of Singapore from 2021 to 2022. He is currently an Associate Professor at the School of Mechanical Engineering, University of Science and Technology Beijing. His research interests include reinforcement learning, optimal control, and autonomous driving decision-making.